一、C语言介绍
二、第一个C程序
1、编译执行
2、详解HelloWorld
三、编译器和gcc
1、什么是编译器
2、什么是gcc
3、gcc编译器把C代码变成可执行程序的过程
4、gcc编译器常用的参数
5、C语言的文件类型
四、C语言中基本的数据类型
1、为什么要把数据分为不同的类型
2、整型
有符号整型：
无符号整型：
3、浮点型
4、字符型
5、bool 布尔型
五、变量
1、什么是变量
2、定义变量
3、变量的取名规则
4、特殊的合法的变量名
5、非法变量名举例
6、变量的使用
7、变量的输出
8、变量的输入
六、常量
七、运算符
1、运算符优先级
2、算术运算符
注意1：
注意2：
3、关系运算符
注意1：
注意2：
注意3：
4、逻辑运算符
短路特性：
5、自变运算符
前自变：++/--i
后自变：i++/--
注意1：
注意2：
6、赋值运算符及扩展
7、三目运算符
注意：
8、字节数运算符
注意：
八、类型转换
1、自动类型转换(隐式类型转换)
2、强制类型转换
九、分支语句
1、if语句
2、switch语句

3、gnu编译器的专用语法
十、循环语句
1、for循环语句
for循环的不同写法：
2、while循环语句
3、do while循环语句
5、循环嵌套
十一、跳转语句
1、break跳转语句
2、continue语句
3、goto语句
4、return语句
十二、数组
1、什么是数组
2、如何定义数组
3、访问数组中的变量
4、遍历数组
5、数组的初始化
6、数组越界
7、数据越界的后果
十三、二维数组
1、什么是二维数组
2、定义二维数组
3、访问二维数组元素
4、二维数组的遍历
5、二维数组初始化
十四、变长数组
1、定长数组
2、变长数组
3、变长数组优缺点
十五、函数
1、什么是函数
2、函数的分类
标准库函数：
系统函数：
第三方库函数：
自定义函数：
3、常用标准库函数介绍：
stdio.h 输入输出相关功能的函数：
stdlib.h 实用的库函数：
ctype.h 字符类型的判断函数：
time.h 时间日期相关的函数：
math.h 数学相关的函数：
4、自定义函数
函数声明：
函数定义：
函数调用：
自定义函数要注意的问题：

一、C语言介绍

丹尼斯.里奇和肯.汤普逊于1971~1973年在贝尔实验室，在开发UNIX操作系统时，以BCPL语言为基础研发了一款高级编程语言，它是为了开发操作系统而研发的一款编程语言，它特别擅长控制硬件，所以在服务器、驱动编程、单片机、嵌入式使用较多。

优点：语法简单，只有32个关键字，执行速度快能媲美汇编语言的执行速度，也适合用于实现算法。

缺点：学习难度大，需要对内存、操作系统有一定的了解，由于出现的较早个人计算机还没有普及，所以在设计时没有为普通人考虑太多，有一些语法上的陷阱和缺陷，没有大型的软件公司在背后支持，可用的软件库比较少。

二、第一个C程序

```
// 输出Hello World
#include <stdio.h>

int main(int argc,const char* argv[])
{
    printf("Hello World!\n");
    return 0;
}
```

1、编译执行

```
gcc hello.c 如果代码没有错误，会生成a.out可执行程序。
./a.out 执行程序
```

2、详解HelloWorld

1、程序员所编写的代码并不是标准C代码，它需要一段程序把它翻译成标准的C代码，负责翻译的程序叫预处理器，被翻译的代码叫预处理指令，所有的预处理指令都是以#开头。

2、#include 就是预处理指令，它的功能是导入一个辅助文件到当前代码。

```
#include "filename.h" 用于导入自定义的头文件
#include <filename.h> 用于导入系统提供的头文件(标准库、操作系统)
```

3、C语言标准委员会为C语言以函数形式提供了一些基础功能，这些函数被封装在libc.so库文件中，并且提供了一些辅助说明文件，常用的有：stdio.h stdlib.h string.h stdint.h等等。

4、stdio.h头文件是 stdandard input output里面对输入输入数据的函数进行了说明，当我们在代码需要使用输入、输出数据的函数时，就需要包含该文件。

5、C语言中函数是管理代码的最小单位，一个函数就一段具有某项功能的代码，main函数是C程序的执行入口，有且仅有一个，无论它定在任何位置都最先执行。

6、函数名后面的内容是函数调用者传递给函数的数据，在命令行执行程序时，后面可以附带一些数据传递给main函数。

7、函数前面的内容是一种数据类型，它表示函数的执行结果是什么类型的数据，int指的就是整型。

8、C语言中使用大括号划分代码区域，被大括号包含的代码都属于main函数的函数体。

9、printf/scanf是标准库的函数，用于输出、输入数据，一般用于调试程序。

```
// printf在输出数据会有一些不方便输入的字符有特殊的字符表示(转义字符):
    \n 换行
    \r 回到行首
    \b 退格
    \t 制表符
    %% 输出一个%d
    \\ 输出一个\
```

10、return语句有两项功能，结束函数的执行，当执行到retunr语句时，那怕后面还有代码都不会再执行了，另外可以返回一个数据给函数的调用者。

11、main函数的返回值表示了程序的结束状态：
return 正数，表示程序执行出现异常。
return 0，表示程序正常结束。
return 负数，表示程序执行出现错误。
echo $? 查看main函数的返回值

12、C代码中以分号作为一行代码的结束标志，如果代码过长可以换行。

三、编译器和gcc

1、什么是编译器

它是一个负责翻译代码的程序，它负责把人类能看懂的代码翻译成计算机能理解的二进制指令，它由预处理器、编译器、汇编器、链接器组成，统称为编译器。

2、什么是gcc

gcc是GNU组织为了编译Linux内核代码而开发一款C语言编译器。

3、gcc编译器把C代码变成可执行程序的过程

```
1、把程序员所编写的代码进行预处理
gcc -E hello.c 把预处理的结果显示到屏幕上
gcc -E hello.c -o hello.i 会生成以.i结尾的预处理文件

2、把预处理的结果翻译成汇编代码
gcc -S hello.i 会生成以.s结尾的汇编文件

3、把汇编代码翻译成二进制指令
gcc -c hello.s 会生成以.o结尾的目标文件

4、把若干个文件目标文件、库文件合并成可执行文件
gcc a.o b.o c.o ... 默认会生成a.out可执行文件，也可以使用-o指定可执行文件的名字。
gcc a.o b.o c.o -o hello

5、执行程序
./hello

6、编译并执行
gcc a.o b.o c.o && ./a.out
```

gcc hello.c 就包含了以上四个步骤，了解这个步骤是我们后续学习预处理指令、多文件编程、静态库、动态库的基础。

4、gcc编译器常用的参数

```
    -E 预处理
    -S 生成汇编文件
    -c 生成目标文件
    -o 设置编译结果的名字
    -I 设置要导入的头文件的路径
    -l 设置要链接的库名，例如：使用sqrt、pow等数学函数时就需要链接数学库 -lm
    -L 设置要链接的库的路径
    -D 在编译时定义宏
```

```
    -g 编译时添加调试信息，这样编译出的程序可以用gdb调试。
    -Wall 显示所有警告，编译器会以更严格的标准检查代码
    -Werror 把警告当错误处理
    -std 指定编译器要遵循的语法标准，c89,c99,c11，当前系统默认的是c99标准。

#为了以后方便执行代码，我们给vim设置一个编译执行的快捷键,~/.vimrc，在文件的末尾添加以下内容。
    " 自定义一个编译运行代码的函数
    func CompileRun()
        " 保存代码
        exec "w"
        " 判断文件的扩展名是否是.c
        if &filetype == "c"
            " 执行编译代码并执行程序的指令
            exec "!gcc % -lm && ./a.out"
        endif
    endfunc

    " 映射正常模式下的Ctrl+x快捷键，调用编译运行函数
    map <C-x> :call CompileRun() <CR>
    " 映射插入模式下的Ctrl+x快捷键，调用编译运行函数
    imap <C-x> <ESC>:call CompileRun() <CR>

# vim新建.c文件时，让vim自动填充头文件导入和main函数，~/.vimrc，在文件的末尾添加以下内容:
    " 定义填充头文件、main的函数
    func InsertMain()
        call setline(1,"#include <stdio.h>")
        call setline(2,"")
        call setline(3,"int main(int argc,const char* argv[])")
        call setline(4,"{")
        call setline(5,"\t")
        call setline(6,"\treturn 0;")
        call setline(7,"}")
        exec "5"
    endfunc
    " 当创建新的.c文件时，自动调用InsertMain函数
    autocmd BufNewFile *.c :call InsertMain()
```

5、C语言的文件类型

```
    .h 头文件，里面是一些对.c文件的说明
    .c 源文件，里面是一些功能型代码
    .i 预处理文件
    .s 汇编文件
    .o 目标文件
    .gch 头文件的编译结果，用于检查自定义的头文件是否有语法错误，建议立即删除
    .a 静态库文件
    .so 动态库文件
```

四、C语言中基本的数据类型

1、为什么要把数据分为不同的类型

数据存储计算机中需要耗费存储空间(内存、硬盘)，在编程语言中把数据按照使用范围、特点划分为不同的种类，解决什么问题使用什么类型的数据，这样可以节约存储空间、提高运算速度。

2、整型

有符号整型：

最高位的二进制用于表示正负，0代表正数，1代表负数。
由于有符号整型使用较多，所以编译器默认signed可以省略，不加就代表加。

```
类型名            占用内存字节数      取值范围
signed char          1        -128 ~ 127
signed short         2        -32768 ~ 32767
signed int           4        -2147483648 ~ 2147483647
signed long          4|8
signed long long     8        -9223372036854775808 ~ 9223372036854775807
```

无符号整型：

所有的二进制位都用来表示数据，只能表示正数。

```
类型名            占用内存字节数  最大值
unsigned char        1        255
unsigned short       2        65535
unsigned int         4        4294967295
unsigned long        4|8
unsigned long long   8        18446744073709551615

无符号类型一般用于计数，unsigned 不能省略，使用起来比较麻烦，标准库为了让我们方便使用，在stdint.h 头文件中对它们进行的类型重定义，分别是：
        uint8_t uint16_t uint32_t uint64_t size_t

```

3、浮点型

小数点是浮动的，也就是带小数点的数据，采用科学计算数法存储，由符号位+指数域+小数域组成，这种存储格式，运算速度慢、又不太准确，所以尽量少用。

```
单精度：  float          4
双精度：  double         8
高精度：  long double    12|16
注意：小数点后六位有效。
```

4、字符型

char 字符型，字符就是符号或图案，但在计算机中以整数形式存储（使用整数模拟字符），当需要显示时根据ASCII表中的对应关系显出相应的符号或图案。

ASCII表

(American Standard Code for Information Interchange 美国标准信息交换代码)

高四位 / 低四位		ASCII控制字符												ASCII打印字符															
		0000						0001						0010		0011		0100		0101		0110		0111		
		0						1						2		3		4		5		6		7		
		十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	Ctrl
0000	0	0		^@	NUL	\0	空字符	16	▶	^P	DLE		数据链路转义	32		48	0	64	@	80	P	96	`	112	p	
0001	1	1	☺	^A	SOH		标题开始	17	◀	^Q	DC1		设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q	
0010	2	2	☻	^B	STX		正文开始	18	↕	^R	DC2		设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r	
0011	3	3	♥	^C	ETX		正文结束	19	‼	^S	DC3		设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s	
0100	4	4	♦	^D	EOT		传输结束	20	¶	^T	DC4		设备控制 4	36	$	52	4	68	D	84	T	100	d	116	t	
0101	5	5	♣	^E	ENQ		查询	21	§	^U	NAK		否定应答	37	%	53	5	69	E	85	U	101	e	117	u	
0110	6	6	♠	^F	ACK		肯定应答	22	▬	^V	SYN		同步空闲	38	&	54	6	70	F	86	V	102	f	118	v	
0111	7	7	•	^G	BEL	\a	响铃	23	↨	^W	ETB		传输块结束	39	'	55	7	71	G	87	W	103	g	119	w	
1000	8	8	◘	^H	BS	\b	退格	24	↑	^X	CAN		取消	40	(	56	8	72	H	88	X	104	h	120	x	
1001	9	9	○	^I	HT	\t	横向制表	25	↓	^Y	EM		介质结束	41	)	57	9	73	I	89	Y	105	i	121	y	
1010	A	10	◙	^J	LF	\n	换行	26	→	^Z	SUB		替代	42	*	58	:	74	J	90	Z	106	j	122	z	
1011	B	11	♂	^K	VT	\v	纵向制表	27	←	^[	ESC	\e	溢出	43	+	59	;	75	K	91	[	107	k	123	{	
1100	C	12	♀	^L	FF	\f	换页	28	∟	^\	FS		文件分隔符	44	,	60	<	76	L	92	\	108	l	124	|	
1101	D	13	♪	^M	CR	\r	回车	29	↔	^]	GS		组分隔符	45	-	61	=	77	M	93	]	109	m	125	}	
1110	E	14	♫	^N	SO		移出	30	▲	^^	RS		记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~	
1111	F	15	☼	^O	SI		移入	31	▼	^-	US		单元分隔符	47	/	63	?	79	O	95	_	111	o	127	⌂	^Backspace 代码：DEL

注：表中的ASCII字符可以用“Alt + 小键盘上的数字键”方法输入。 2013/08/08

5、bool 布尔型

C语言中没有真正的布尔类型，因为先有的C语言后出现的布类型，在C89之后以打补丁的方式新增布尔类型，需要包含stdbool.h头文件。

```
bool     1字节
true     4字节  实际使用的是整数1
false    4字节  实际使用的是整数0
```

注意：复合数据类型结构、联合、枚举、指针，后期会详细讲解。

五、变量

1、什么是变量

在程序运行过程可以变化的量，它是存储数据的容器，需要先定义才能使用。

2、定义变量

> 数据类型 <变量名>;

1、变量的所占用的内存字节数、存储数据的范围、使用的规则都由变量的类型决定。

2、变量的定义就是操作系统把一个标识符与内存建议映射关系，操作系统不会对这个内存进行初始化，所以变量的默认值是不确定的，我们要对一些特殊用途的变量进行初始化，比如：计数、求和、平均、累加。

3、定义的变量，出了它所在的大括号就不能再使用了。

3、变量的取名规则

1、由数字、字母、下划线组成。
2、不能以数字开头。
3、不能与关键字重名(C语言中有32个关键字)。
4、见名知义，类型、功能、使用范围、归属模块。

关键字	释义	关键字	释义
auto	声明自动变量	int	声明整型变量或函数
break	跳出当前循环	long	声明长整型变量或函数返回值类型
case	开关语句分支	register	声明寄存器变量
char	声明字符型变量或函数返回值类型	return	子程序返回语句（可以带参数，也可不带参数）
const	声明只读变量	short	声明短整型变量或函数
continue	结束当前循环，开始下一轮循环	signed	声明有符号类型变量或函数
default	开关语句中的"默认"分支	sizeof	计算数据类型或变量长度（即所占字节数）
do	循环语句的循环体	static	声明静态变量
double	声明双精度浮点型变量或函数返回值类型	struct	声明结构体类型
else	条件语句否定分支（与 if 连用）	switch	用于开关语句
enum	声明枚举类型	typedef	用以给数据类型取别名
extern	声明变量或函数	unsigned	声明无符号类型变量或函数
float	声明浮点型变量或函数返回值类型	union	声明共用体类型
for	一种循环语句	void	声明函数无返回值或无参数，声明无类型指针
goto	无条件跳转语句	volatile	说明变量在程序执行中可被隐含地改变
if	条件语句	while	循环语句的循环条件

4、特殊的合法的变量名

bool，printf，true，flase

5、非法变量名举例

1num、*num、auto

6、变量的使用

num = 0;　// 被赋值、存储数据，相当是个容器
num*3/4;　// 参与运算，此时变量名就代表它存储的数据

7、变量的输出

在实际项目中变量没有输入、输出的需求，但在调试阶段，我们为了检查程序的运行是否正确，一般使用printf函数输出变量的值，模拟把数据显示在界面上，使用scanf输入变量的值，模拟从界面获取用户输入的数据。

```
#include <stdio.h>

int printf(const char *format, ...);
功能：它是C标准库函数，把若干数据输出到终端上
format：提示信息+变量的类型信息+转义字符
...：若干个变量名，变量名之间有逗号分隔
返回值：printf输出到终端上的字符个数

在C语言中使用占位符来表示变量的类型：
    %hhd %hd %d %ld %lld　有符号整型的占位符
    %hhu %hu %u %lu %llu　无符号整型的占位符
    %f %lf %Lf　浮点型的占位符
    %c 字符型的占位符
    布尔类型是用来逻辑运算的，不参与输入、输出，如果要强行输出，把它当整型即可。
```

8、变量的输入

```
int scanf(const char *format, ...);
功能：从终端读取数据赋值给变量
format：占位符，提示信息和转义字符不要使用
...：若干个变量的地址，&变量名可以计算出变量的地址
返回值：成功读取数据的变量个数

错误的使用案例：
    scanf("%d\n",&num);
```

练习：定义各类型的变量，使用scanf输入变量的值，使用printf输入变量的值和变量的内存字节数，sizeof(变量名) 得到变量的内存字节数。

```
#include <stdio.h>

int main()
{
    char hhd_num;
    short hd_num;
    int d_num;
    long ld_num;
    long long lld_num;

    unsigned char hhu_num;
    unsigned short hu_num;
    unsigned int u_num;
    unsigned long lu_num;
    unsigned long long llu_num;

    float f_num;
    double lf_num;
    long double Lf_num;

    scanf("%hhd %hd %d %ld %lld %hhu %hu %u %lu %llu %f %lf %Lf",
            &hhd_num,&hd_num,&d_num,&ld_num,&lld_num,
            &hhu_num,&hu_num,&u_num,&lu_num,&llu_num,
            &f_num,&lf_num,&Lf_num);
    printf("%hhd %hd %d %ld %lld %hhu %hu %u %lu %llu %f %lf %Lf",
            hhd_num,hd_num,d_num,ld_num,lld_num,
            hhu_num,hu_num,u_num,lu_num,llu_num,
            f_num,lf_num,Lf_num);
}
```

六、常量

常量就是程序运行过程中不能改变的量，C语言的常量有：字面值常量、宏常量、枚举常量。

Character and Character String Literals

Prefix	Meaning	Type
u	Unicode 16 character	char16_t
U	Unicode 32 character	char32_t
L	wide character	wchar_t
u8	utf-8 (string literals only)	char

Integer Literals		Floating-Point Literals	
Suffix	**Minimum Type**	**Suffix**	**Type**
u or U	unsigned	f or F	float
l or L	long	l or L	long double
ll or LL	long long		

```
100     int
100l    signed long
100ll   signed long long
100u    unsigned int
100lu   unsigned long
100llu  unsigned long long

3.14    double
3.14f   float
3.14F   long double
'A'     char
```

七、运算符

1、运算符优先级

优先级	运算符	名称或含义	使用形式	结合方向	说明
1	[]	数组下标	数组名[常量表达式]	左到右	
	()	圆括号	(表达式) 函数名(形参表)		
	.	成员选择（对象）	对象.成员名		
	->	成员选择（指针）	对象指针->成员名		
2	-	负号运算符	-表达式	右到左	单目运算符
	(类型)	强制类型转换	(数据类型)表达式		
	++	自增运算符	++变量名 变量名++		单目运算符
	--	自减运算符	--变量名 变量名--		单目运算符
	*	取值运算符	*指针变量		单目运算符
	&	取地址运算符	&变量名		单目运算符
	!	逻辑非运算符	!表达式		单目运算符
	~	按位取反运算符	~表达式		单目运算符
	sizeof	长度运算符	sizeof(表达式)		
3	/	除	表达式 / 表达式	左到右	双目运算符
	*	乘	表达式*表达式		双目运算符
	%	余数（取模）	整型表达式%整型表达式		双目运算符
4	+	加	表达式+表达式	左到右	双目运算符
	-	减	表达式-表达式		双目运算符
5	<<	左移	变量<<表达式	左到右	双目运算符
	>>	右移	变量>>表达式		双目运算符
6	>	大于	表达式>表达式	左到右	双目运算符
	>=	大于等于	表达式>=表达式		双目运算符
	<	小于	表达式<表达式		双目运算符
	<=	小于等于	表达式<=表达式		双目运算符
7	==	等于	表达式==表达式	左到右	双目运算符
	!=	不等于	表达式!= 表达式		双目运算符
8	&	按位与	表达式&表达式	左到右	双目运算符
9	^	按位异或	表达式^表达式	左到右	双目运算符

10	|	按位或	表达式|表达式	左到右	双目运算符
11	&&&	逻辑与	表达式&&表达式	左到右	双目运算符
12	||	逻辑或	表达式||表达式	左到右	双目运算符
13	?:	条件运算符	表达式1? 表达式2: 表达式3	右到左	三目运算符
14	=	赋值运算符	变量=表达式	右到左	
	/=	除后赋值	变量/=表达式		
	*=	乘后赋值	变量*=表达式		
	%=	取模后赋值	变量%=表达式		
	+=	加后赋值	变量+=表达式		
	-=	减后赋值	变量-=表达式		
	<<=	左移后赋值	变量<<=表达式		
	>>=	右移后赋值	变量>>=表达式		
	&=	按位与后赋值	变量&=表达式		
	^=	按位异或后赋值	变量^=表达式		
	|=	按位或后赋值	变量|=表达式		
15	,	逗号运算符	表达式,表达式,...	左到右	

2、算术运算符

```
+    加
-    减
*    乘
/    除     进行除法运算得到的结果是商
%    求余  进行除法运算得到的结果是余数
```

注意1：

整数/整数 计算的结果没有小数点，例如：3/5 的结果是1。

注意2：

/和% 都是进行除法运算，/结果是商，%结果是余数，它们除数不以为零，会出现浮点数例外的错误，该错误会让程序立即结束。

3、关系运算符

```
>    大于
>=   大于等于
<    小于
<=   小于等于
==   相等
!=   不相等
```

注意1：

它们运算结果是逻辑值，C语言中的逻辑值是用0(假)和1(真)模拟的，计算出的结果还能进行数学运算。

注意2：

与数学中的用法不同，10 < x < 100，在数学中表示是x的取值范围，但在C语言中它就是需要运算的表达式，会先计算 10 < x 得到0|1，然后再把这个结果与100比较，所以该表达的结果永远是真。

注意3：

使用==运算符时，非常容易出错误，容易漏写一个=，该错误编译器检查不出，阅读代码也很难查出来，所以在使用==时，把常量放在左边，变量写右边，这样当漏写=时，编译器会报错误。

```c
int num;
if(num = 100)    // 变成了赋值语句，num被赋值为100，并且if分支永远执行
{

}

if(100 = num)    // 用变量给常量赋值，编译器会报错，
{

}
```

4、逻辑运算符

```
&&  与  双目运算符，左右两边都是真，结果才是真
||  或  双目运算符，左右两边只要有一个是真，结果就是真
!   非  单运算符，只针对右边的运算对象，求反，如果运算对象是真，结果就是假，如果运算对象是假，结果就是真
注意：!比&&、||的运算优先级高。
```

它们会把运算对象转换成逻辑值再运算，零值转换为假，非零值转换为真，它们的运算结果也就是逻辑值。

短路特性：

当左边的值已经可以确定运算结果时，右边的不再计算，适当的使用可以实现精简的if结构，能看懂即可，不要这样写，因为它会影响代码的可读性。

```
(表达式1) && (表达式2) 如果表达式1结果是假，表达式2不再计算。
(表达式1) || (表达式2) 如果表达式1结果是真，表达式2不再计算。

num >10 && (num=0); // 等价于下面的if语句
if(num > 10)
{
    num = 0;
}
```

5、自变运算符

前自变：++/--i

变量的值立即加1或减1。

后自变：i++/--

变量的值加1或减1，下一行代码有效。

注意1：

该类运算符可以让变量的值自加或自减1，但只能针对变量使用。

注意2：

不要在复杂表达式中过度使用自变运算符，不同的编译器对它的运算规则不同，编译器会把合适的后自变优化成前自变。

6、赋值运算符及扩展

```
= += *= /= %=... 是变量先运行后赋值的一种精简写法
a += b; 等价  a = a + b;
a *= b; 等价  a = a * b;
```

7、三目运算符

`? :;`

它的运算对象有三个，所以叫三目运算符，先把转换成逻辑值，为真执行，为假执行，相当于精简的 if else 结构。

注意：

与 if else 不同的是三目运算符必须有运算结果，所以它里面不能使用流程控制语句，比如：return、break、continue。

8、字节数运算符

sizeof它不是函数，是C语言的32个关键字之一，它能计算出数据在内存需要占用的内存字节数，如果运算对象不是表达式，它可以不使用小括号。

注意：

sizeof(表达式)不会执行小括号里表达式，它只时推算表达式的执行结果是什么类型，然后计算出该类型在内存中占用多少个字节。

八、类型转换

> 前提：只有相同类型的数据才能在一起进行运算，因为不同类型的数据，字节数不同、格式、运算规则不同，必须把不同类型的数据转换成同一类型才能运算。

1、自动类型转换(隐式类型转换)

不同类型的数据组成的表达式，编译器会把先它们转换成相同的类型再计算，这叫作自动类型类型或隐式类型转换，它们的转换规则是以不丢失数据为前提：

1、字节数不同，字节少的向多的转换。
2、整型向浮点型转换。
3、有符号的向无符号转换。
4、char、short会先转换成int类型，如果不能满足运算条件再转换成其它。

2、强制类型转换

(目标类型)数据，会把数据强制转换为目标类型，这种转换方式有可能会造成数据丢失，慎重使用。

九、分支语句

1、if语句

代码的默认执行流程是从上到下，逐步、逐条执行的，if语句可以根据判断条件选择让代码是否执行，改变了代码 的默认执行流程，所以这种语句也叫流程控制语句。

```
if(条件) // 单分支
{
    // 当条件为真时，执行此处代码，如果此处代码只有一行，大括号可以省略，但在商业项目中建议不要省略，因为这样会影响代码的可扩展性
}

if(条件)  // 双分支
{
    // 当条件为真时，执行此处代码
}
else
{
    // 当条件为假时，执行此处代码
}

if(条件1)
{
    // 当条件1为真时，执行此处代码
}
else if(条件2) // 可以有多个else if
{
    // 当条件2为真时，执行此处代码
}
else
{
    // 当条件1、条件2都为假时，执行此处代码
}
```

练习1、输入一个整数，判断是奇数还是偶数。

```c
#include <stdio.h>

int main()
{
    int num;
    printf("请输入一个整数：");
    scanf("%d",&num);
    if(num%2)
    {
        printf("奇数\n");
    }

    else
    {
        printf("偶数\n");
    }
}
```

练习2、输入一个年份，判断是闰年还是平年。

```c
#include <stdio.h>
// 四闰百不闰，四百闰
int main()
{
    short year;
    printf("请输入一个年份：");
    scanf("%hd",&year);
    if(0==year%4 && 0!=year%100 || 0==year%400)
    {
        printf("闰年!\n");
    }
    else
    {

        printf("平年\n");
    }
}
```

练习3、输入一个月份，判断该月有多少天。

```c
#include <stdio.h>

int main()
{
    char month;
    printf("请输入一个月份:");
    scanf("%hhd",&month);

    if(0 >= month || 12 < month)
    {
        printf("输入的月份有误！\n");
    }
    else if(2 == month)
    {
        printf("28天\n");
    }
```

```
    else if(4==month || 6==month || 9==month || 11==month)
    {
        printf("30天\n");
    }
    else
    {
        printf("31天\n");
    }
}
```

2、switch语句

1、switch小括号中的数据可以是变量或表达式，但结果必须是整数型，case后面的数据必须是整数型常量。

2、把switch小括号中的数据与case后的数据进行对比，如果相同则打开执行开关。

3、当执行开关处于打开状态时，case后面的语句就会被执行，如果执行开关不关闭，之后case、default后面的语句也会被执行。

4、break语句可以关闭执行开关，如果每个case后都break语句，就形成了多分支结构。

5、default无论写在任何位置，都是等所有的case没有匹配成功后，才打开执行开开关，就相当于多分支if语句最后的else，可以有也可以没有。

```
switch (数据)
{
    case v1: 语句1; break;
    case v2: 语句2; break;
    case v3: 语句3; break;
    ...
    default: 语句; break;
}
```

练习：输入一个月份，显示出该月有多少天，忽略闰年。

```
#include <stdio.h>

int main()
{
    char month;
    printf("请输入一个月份：");
    scanf("%hhd",&month);
    if(0 >= month || month > 12)
    {
        printf("请输入月份有误!\n");
        return 0;
    }

    switch(month)
    {
        case 2:
            printf("28天\n"); break;
        case 4: case 6: case 9: case 11:
            printf("30天\n"); break;
        default:
           printf("31天\n");
```

```
    }
}
```

3、gnu编译器的专用语法

```
switch(整型数据)
{
case start1 ... end1: 语句1; break;
case start2 ... end2: 语句2; break;
case start3 ... end3: 语句3; break;
}
```

十、循环语句

1、for循环语句

通过反复执行一段代码，达到解决问题的目的，for是一种非常灵活的循环，一般都使用一个变量来引导它的执行过程，这个变量被称为循环变量，早期使用index作为循环变量名，然后逐渐演变成i，如果有多个循环也可以使用j、k、l。

```
for([1]; [2]; [3])
{
    [4];
}

1、为for循环做一些准备工作，定义循环变量、给循环变量赋初值，C89的语法标准下不能定义变量，在此处定义的变量和出了for循环就不能再使用。
2、判断循环条件，条件为真执行4，条件为假结束for循环，如果没有语句，则默认为真
4、被反复执行的代码也叫循环体
3、改循环条件，循环变量自加或自减1，防止变成死循环

循环流程：
    1 2 4 3 2 4 3 2 4 3 ... 2
```

for循环的不同写法：

```
for(;;) // 死循环
{
    ...
}

int i=0;
for(; i<10; i++)
{
    ...
}

int i=0;
for(; i<10;)
{
    ...
    i++;
}

for(int i=0; i<10; i++,...);
```

练习1、输入两整数，计算出它们的最大公约数、和最小公倍数。

```c
#include <stdio.h>

int main()
{
    int num1,num2,max=1,min;
    printf("请输入两个整数：");
    scanf("%d%d",&num1,&num2);
    if(num1 > num2)
    {
        int tmp = num1;
        num1 = num2;
        num2 = tmp;
    }

    for(int i=2; i<=num1; i++)
    {
        if(0 == num1%i && 0 == num2%i)
        {
            max = i;
        }
    }

    /*
    for(int i=num1*num2; i>=num2; i--)
    {
        if(0==i%num1 && 0 == i%num2)
        {
            min = i;
        }
    }*/

    printf("%d %d\n",max,num1*num2/max);
}
```

练习2、输入前n项斐波那切数列，1 0 1 1 2 3 5 8 13 21...

```c
#include <stdio.h>


int main()
{
    int n, num1 = 1,num2 = 0;
    printf("请输入n的值：");
    scanf("%d",&n);

    for(int i=1; i<=n; i++)
    {
        printf("%d ",num1+num2);
        num2 = num1+num2;
        num1 = num2-num1;
    }
```

```
}
```

2、while循环语句

```
while(循环条件)
{
    // 循环体
}
```

执行流程：先检查循环条件，为真则执行循环体，为假则结束while循环。

如果for循环语句忽略模块1和模块3功能与while一模一样，它相当于for循环的精简版本，for适合解决循环次数明确的问题，while适合解决只知道循环条件，但不确定具体的循环次数的问题。

练习1、输入一个正整数，倒序显示，例如：输入12345 显示54321。

```
#include <stdio.h>

int main()
{
    unsigned int num;
    printf("请输入一个正整数：");
    scanf("%u",&num);

    while(num)
    {
        printf("%d",num%10);
        num /= 10;

    }
}
```

练习2、输入一个整数，倒序显示，例如：输入-12345 显示-54321。

```
#include <stdio.h>

int main()
{
    int num;
    printf("请输入一个正整数：");
    scanf("%d",&num);
    if(num < 0)
    {
        printf("%d",num%10);
        num /= 10;
        num *= -1;

    }

    while(num)
    {
        printf("%d",num%10);
        num /= 10;
    }
}
```

3、do while循环语句

```
do {
        // 循环体
}while(循环条件);
```

执行流程：先执行循环体，再判断循环条件，当条件为真时继续执行循环体，当条件为假时结束循环。

无论条件真假，循环体至少执行一次，相同条件下，它会比for和while多执行一次循环体。

注意：在 do while 循环体中定义的变量，不能在小括号中使用，类似for循环中定义的循环变量，出了大括号也不能使用。

练习、输入密码(123234)，判断是否正确，如果正确则显示登录成功，如果密码错误则显示剩余机会，超过三次则显示账号锁定。

```
#include <stdio.h>

int main()
{
    int passwd;

    for(int cnt=0;123234!=passwd && cnt<3; cnt++)
    {
        if(0 == cnt)
            printf("请输入密码：");
        else
            printf("密码错误，请重新输入密码(还有%d次机会)",3-cnt);
        scanf("%d",&passwd);
    }

    if(123234 == passwd)
        printf("登录成功！");
    else
        printf("账号已锁定!\n");
}
```

5、循环嵌套

循环语句中包含循环语句，外层循环执行一次，内层循环执行一遍。

```
for(int i=0; i<10; i++)
{
    for(int j=0; j<10; j++)
    {
        printf("*");
    }
    printf("%d\n",i);
}
```

十一、跳转语句

1、break跳转语句

break 语句有两种用法：
用法1：在 switch 语句中可以关闭执行开关。
用法2：在循环语句中，它可以跳出一层循环，也是结束死循环的一种方式，一般用它来提前结束循环，提高程序的执行效率。

```
#include <stdio.h>

int main()
{
    int i = 0;
    for(;;)
    {
        if(i>=10)
            break;
        printf("%d\n",i++);
    }
}
```

2、continue语句

只能在循环语句中使用，停止本次循环体的执行，直接进行下一次循环，一般用于改善大括号的嵌套层数。

```
#include <stdio.h>

int main()

{
    for(int i=0; i<10; i++)
    {
        if(i%2)
            continue;

        printf("%d\n",i++);
    }
}
```

3、goto语句

goto 语句可以跳转到函数内的任意位置，由于它过度灵活、自由可能会破坏已经设计好的分支、循环语句，因此一般公司都禁止使用，新的编程语句中已经取消该关键字。

goto非常适合在驱动程序中处理异常、释放资源，所以C语言中从语法设计上就非常适合控制硬件。
在函数内通过定义标签确定代码位置。

```
#include <stdio.h>

int main()
{
    int i = 0;
    goto line3; // 跳转到某个位置标签
```

```
// 定义位置标签
line1:
    printf("1\n");
line2:
    printf("2\n");
line3:
    printf("3\n");
line4:
    printf("4\n");
line5:
    printf("5\n");
}
```

4、return语句

return 语句可以跳出函数，并附加一个数据给函数的调用者。

```
#include <stdio.h>

int main()
{
    for(int i=0; i<10; i++)
    {
        if(5 == i)
            return 0;

        printf("%d\n",i);
    }
    printf("over!\n");
}
```

练习1、打印九九乘法表。

```c
#include <stdio.h>

int main()
{
    for(int i=1; i<=9; i++)
    {
        for(int j=1; j<=i; j++)
        {
            printf("%dx%d=%d ",j,i,j*i);

        }
        printf("\n");
    }
}
```

2、计算出100到1000之间所有的素数，每7个一行。

```c
#include <stdio.h>

int main()
{
    int cnt = 0;
    for(int i=100,j; i<1000; i++)
    {
        for(j=2; j<i; j++)
        {
            if(0 == i%j)
                break;
        }
        if(j == i)
        {
            printf("%d ",i);
            if(0 == ++cnt % 7)
                printf("\n");
        }


    }
}
```

3、计算出10000以内所有的完美数，因子之和等于这个数本身这种整数叫完美数，例如：6=1+2+3。

```c
#include <stdio.h>

int main()
{
    for(int i=1; i<10000; i++)
    {
        int sum = 0;
        for(int j=1; j<=i/2; j++)
        {
            if(0 == i%j)
                sum += j;
        }
        if(i == sum)
```

```
            printf("%d ",i);

    }
}
```

十二、数组

1、什么是数组

数组就是变量的组合，是一种批量定义变量的方式。

2、如何定义数组

```
类型 数组名[数量];
int arr[8]; 就相当于定义了8个变量 int b1,b2,b3,b4,b5,b6,b7,b8;
```

3、访问数组中的变量

数组名[下标];
下标就是变量的编号，下标的范围：0 , 数量-1

4、遍历数组

与for循环配合，使用循环变量作为数组的下标。

```
int arr[8];
for(int i=0; i<8; i++)
{
    printf("%d ",arr[i]);
}
```

5、数组的初始化

数组元素的默认值与普通变量一样，是随机的，如果需要对数组初始化可以使用以下语法。
类型 数组名[数量] = {v1,v2,v3,...};

1、如果初始化数据不够，编译器会自动补0。
2、如果初始化数据过多，编译器会提示警告并丢弃多余数据。
3、对数组进行初始化时，数组的数量可以省略，编译器会自动统计数据的数量设置给数组。
4、初始化语法只能在定义数组时使用，这也是唯一一次能对数组批量访问的机会，数组定义完成后就只能单个访问。

```
int arr[] = {1,3,1,2,1,3,2,5,3,5,2,3,6,3};
int len = sizeof(arr)/sizeof(arr[0]);
for(int i=0; i<len; i++)
{
    printf("%d ",arr[i]);
}
```

练习：

1、输入一个整数，分解显示
例1：输入123456 输出1 2 3 4 5 6
例2：输入-123456 输出- 1 2 3 4 5 6

```c
#include <stdio.h>

int main()
{
    int num;
    printf("请输入一个整数：");
    scanf("%d",&num);
    char arr[10],cnt=0;

    if(num < 0)
    {
        printf("- ");
        arr[cnt++] = num % 10 * -1;
        num = num / 10 * -1;
    }

    while(num)
    {
        arr[cnt++] = num % 10;
        num /= 10;
    }

    for(int i=cnt-1; i>=0; i--)
    {
        printf("%d ",arr[i]);
    }
}
```

2、输入一个正整数，判断是否是回文数，例如：123321 12321。

```c
#include <stdio.h>

int main()
{
    unsigned int num;
    printf("请输入一个正整数：");
    scanf("%u",&num);

    unsigned int tmp = num, new = 0;

    while(tmp)
    {
        new = new*10+tmp%10;
        tmp /= 10;
    }

    if(new == num)
        printf("是回文数\n");
    else
        printf("不是回文数\n");
}

#include <stdio.h>

int main()
```

```c
{
    unsigned int num;
    printf("请输入一个正整数：");
    scanf("%u",&num);
    char arr[10],cnt=0;

    while(num)
    {
        arr[cnt++] = num%10;
        num /= 10;
    }

    for(int i=0; i<cnt/2; i++)
    {
        if(arr[i] != arr[cnt-i-1])
        {
            printf("不是回文数\n");
            return 0;
        }
    }
    printf("是回文数\n");
}
```

3、输入一个正整数，计算每个数字出现的次数。

例如：12321

1 出现了2次

2 出现了2次

3 出现了1次

```c
#include <stdio.h>


int main()
{
    unsigned int num;
    printf("请输入一个正整数：");
    scanf("%u",&num);
    char arr[10] = {};

    while(num)
    {
        arr[num%10]++;
        num /= 10;
    }
    for(int i=0; i<10; i++)
    {
        if(arr[i])
        {
            printf("%d 出现了%d次\n",i,arr[i]);
        }
    }
}
```

6、数组越界

使用非法的下标访问数组的行为叫数组越界，当数组越界访问时编译器不会报错，因为使用变量作为数组的下标时，编译器无法检测到数组越界的（程序不运行变量的值无法确定）所以就不再检查数组的下标是否合法，这样做的好处就是代码的编译、运行速度快。

7、数据越界的后果

1、运气好的情况下，会一切正常。

2、段错误，操作系统发现程序非法访问内存，它就会把程序杀死。

3、脏数据，数组越界访问到其它数组、变量的内存位置，破坏了其它数组、变量的数据。

作业1、输入一个日期（yyyy-mm-dd)，计算出这个日期是该年的第 几天。

```c
#include <stdio.h>

int main()
{
    char arr[] = {31,28,31,30,31,30,31,31,30,31,30,31};
    short year,month,day;
    printf("请输入一个日期(yyyy mm dd)：");
    scanf("%hd%hd%hd",&year,&month,&day);

    if(0==year%4 && 0!=year%100 || 0==year%400)
    {
        arr[1] = 29;
    }

    if(year <= 0 || month > 12 || month <= 0 || day <=0 || day >arr[month-1])
    {
        printf("输入的日期有误！\n");
        return 0;
    }

    for(int i=0; i<month-1; i++)
    {
        day += arr[i];
    }


    printf("该日期是%hd年的第%d天\n",year,day);
}
```

作业2、打印以下 图形：

```
/*
    *|
   * |*
  *  | *
 *   |  *
*    |   *
 *   |  *
  *  | *
   * |*
```

```
    *|
*/
#include <stdlib.h>

int main()
{
    for(int i=-4; i<=4; i++)
    {
        for(int j=0; j<abs(i); j++)
        {
            printf(" ");
        }
        printf("*");
        for(int j=0; j<(5-abs(i)-1)*2; j++)
        {
            printf(" ");
        }
        printf("\b*\n");
    }
}
```

作业3、已知rand()函数可以产生一个随机的正整数，实现洗牌的功能。

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    srand(time(NULL));

    char arr[54];
    for(int i=0; i<54; i++)
    {
        arr[i] = i+1;
    }

    for(int i=53; i>0; i--)
    {
        int j = rand() % i;
        int tmp = arr[i];
        arr[i] = arr[j];
        arr[j] = tmp;
    }

    for(int i=0; i<54; i++)
    {
        printf("%d ",arr[i]);
    }
}
```

十三、二维数组

1、什么是二维数组

普通数组也叫一维数组，可以看作是若干个变量排成一排。
二维数组就是若干个row*col个变量排成的方阵。

2、定义二维数组

```
类型 数组名[行数][列数];
int arr[3][5];

// 相当于定义了3行5列15个变量
[] [] [] [] []
[] [] [] [] []
[] [] [] [] []
```

3、访问二维数组元素

数组名 [行下标][列下标];
行下标范围：0 ~ 行数-1
列下标范围：0 ~ 列数-1

```
int arr[3][5];

// 15个变量的行下标和列下标
[0][0] [0][1] [0][2] [0][3] [0][4]
[1][0] [1][1] [1][2] [1][3] [1][4]
[2][0] [2][1] [2][2] [2][3] [2][4]
```

4、二维数组的遍历

需要与双层for循环配合，外层循环变量当作数组的行下标，内层循环变量当作数组的列下标。

```
for(int r=0; r<3; r++)
{
    for(int c=0; c<5; c++)
    {
        printf("%d ",arr[r][c]);
    }
    printf("\n");
}
```

5、二维数组初始化

二维数组在初始化时，其它特点与一维数组相同，不同的是列数必须确定。

```
类型 数组名[行下标][列下标] = {
   {v00,v01,v02,...}, // 第一行
   {v10,v11,v12,...}, // 第二行
   ...
};
```

十四、变长数组

1、定长数组

使用整数常量作为定义数组时的长度参数，或者使用初始化数据省略长度参数，由编译器统计初始化数据的数量设置数组的长度。

定义数组在编译时数组的长度就确定了，这种数组编译器可以帮我们进行初始化操作，为了避免数据不够使用所以在定义时一般要多定义点，可能会浪费一部分内存。

2、变长数组

使用变量作为定义数组的长度参数，而程序不运行变量的值就无法确定，因此在编译时数组的长度就无法确定，编译器也就无法帮我进行初始化操作。
当程序运行起来后，在执行数组的定义语句前，长度变量都可以根据实际情况变化，当执行完数组的定义语句后，数组的长度就固定下来，不会再变了。

```c
int len;
printf("请输入数组的长度:");
scanf("%d",&len);
int arr[len];
```

3、变长数组优缺点

根据本次运行的实际情况，数组需要多长，就定义多长，每次运行都可以不同，避免内存浪费。

不能对变长数组进行初始化。

练习、输入n个的不重复的整数，找出差值最小的两个整数，如果多组则输出任意一组即可。

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n;
    printf("请输入n的值：");
    scanf("%d",&n);
    int arr[n];

    for(int i=0; i<n; i++)
    {
        scanf("%d",&arr[i]);
    }

    int min1 = arr[0] , min2 = arr[1];
    int min = abs(min1-min2);

    for(int i=0; i<n-1; i++)
    {
        for(int j=i+1; j<n; j++)
        {
            if(min > abs(arr[i]-arr[j]))
            {
                min1 = arr[i];
                min2 = arr[j];
                min = abs(min1-min2);
```

```
            }
        }
    }
    printf("%d %d,%d\n",min,min1,min2);
}
```

十五、函数

1、什么是函数

函数就是一段具有某项功能的代码，它是C语言中管理代码的最小单位，把具有某项功能的若干行代码封装在函数中方便管理代码且方便重复调用。

2、函数的分类

标准库函数：

C语言标准委员会为C语言以函数形式提供了一些基础功能，这些函数被封装在libc.so库文件中。

系统函数：

操作系统为程序员提供了一些系统API，可以以函数形式调用，但它们不是真正的函数，讲UNIX系统环境编程时会详细讲解。

第三方库函数：

一些公司或开源组织实现的一些常用工具供程序员使用。

自定义函数：

为了更方便的管理、调用代码，降低开发难度，程序员自己封装的一些函数。

3、常用标准库函数介绍：

标准库中除上封装了函数，还提供一些头文件，里面是对函数的说明。

stdio.h 输入输出相关功能的函数：

```
int printf(const char *format, ...);
功能：输出数组到终端
format：提示信息+占位符+转义字符组成
...：若干个变量名或数据
返回值：成功输出的字符个数

int scanf(const char *format, ...);
功能：从终端读取数据
format：一般情况下，只需要占位符即可，除了占位符以外的信息，在输入数据时要原样补出。
...：若干个变量的地址
返回值：成功读取的变量个数
```

stdlib.h 实用的库函数：

```
int system(const char *command);
功能：调用系统命令，命令执行完成后，该函数才返回
返回值：成功返回0，失败返回-1。

void srand(unsigned int seed);
功能：所谓的随机数就是把所有整数打乱顺序，从某个位置获取，默认从1位置获取，程序运行时如果"位置"不发放，获取随机数与上次一样，为了保证每个运行时，提的位置都发生变化，一般把time函数的返回值提供给srand作为随机数的种子。

int rand(void);
功能：从系统中获取随机数
返回值：都是正整数，如果需要负数或浮点数，需要程序员自已处理。
```

ctype.h 字符类型的判断函数：

```
int isdigit( int ch );
功能：判断是否是数字字符
int islower( int ch );
功能：判断是否是小写字母
int isupper( int ch );
功能：判断是否是大写字母
```

time.h 时间日期相关的函数：

```
time_t time(time_t *tloc);
功能：获取当前系统赶时间，返回自 1970年1月1日 00:00:00 到现在一共过了多少秒，格林时间。

struct tm *localtime(const time_t *timep);
功能：把秒数据时间转换成年月日、时分秒
```

math.h 数学相关的函数：

```
double pow(double x, double y);
功能：计算出x的y次数

double sqrt(double x);
功能：计算x的平方根

double ceil(double x);
功能：向上取整，返回大于x的最小整数

double floor(double x);
功能：向下取整，返回小于x的最大整数
注意：使用这些函数在编译时必须有-lm参数
```

4、自定义函数

有两情况适合把代码封装成自定义函数：

1、代码量过多，一般代码量超过50就要考虑封装成函数，方便管理代码。

2、如果一个代码需要在不同位置多次执行，为了防止出现代码冗余，就要把它封装成函数，方便重复使用。

函数声明：

> 返回值类型 函数名(参数列表);

1、根据函数的功能为函数取名字，在Linux系统下函数全部小写，用下划线分隔。
2、参数列表，指的是函数执行时，调用者需要传递它的数据，此时重点关注的是参数的类型，在函数声明时可以忽略参数的名，如果函数执行时不需要调用者传递数据则写void。
3、返回值类型，指的是函数的执行结果是什么类型数据，如果函数没有返回值，则写void。
4、函数声明就是告诉编译器该函数的格式，方便编译器检查调用者的使用是否正确。

函数定义：

```
返回值类型 函数名(类型 参数名)
{
    // 函数体
}
// 如果函数的定义出现在调用之前，函数声明可以省略
```

函数调用：

> 函数名(实参);

1、调用者会把实参赋值给形参变量。
2、函数的返回值会放置在调用位置，可立即使用，也可用变量保存。

练习1、实现一个判断是否素数的函数，调它打印出100到10000所有的素数。

```
#include <stdbool.h>
#include <math.h>

bool is_prime(int num)
{
    for(int i=2; i<=sqrt(num); i++)
    {
        if(0 == num%i)
            return false;
    }
    return true;
}

int main()
{
    for(int i=100; i<10000; i++)
    {
        if(is_prime(i))
            printf("%d ",i);
    }
}
```

练习2、输入一个日期，计算出它与2008-8-1相隔多少天。

```
#include <stdio.h>


#include <stdlib.h>
#include <stdbool.h>

```

```c
// 判断是否是闰年
bool is_leap(short year)
{
    return 0==year%4 && 0!=year%100 || 0==year%400;
}

// 计算出自公元元年到该日期一共过了多少天
int date_to_days(short year,char month,char day)
{
    int sum = day;

    // 计算整年的天数
    for(int y=1; y<year; y++)
    {
        sum += 365+is_leap(y);
    }

    char arr[12] = {31,28,31,30,31,30,31,31,30,31,30,31};
    arr[1] += is_leap(year);

    // 计算整月的天数
    for(int m=0; m<month-1; m++)
    {
        sum += arr[m];
    }

    return sum;
}

int main()
{
    short year,month,day;
    printf("请输入一个日期(yyyy mm dd): ");
    scanf("%hd%hd%hd",&year,&month,&day);
    printf("该日期与2008-08-01相隔%d天\n",
        abs(date_to_days(2008,8,1)-date_to_days(year,month,day));
}
```

自定义函数要注意的问题:

1、函数的命名空间互相独立，函数之间传参是单身值传递（实参给形参赋值），所以两个函数之间不能通过传参共享变量。

2、C语言中如果函数的参数列表是空的，则意味着该函数提供任意类型、多个参数都可以调用，容易给调用者造成误会，影响代码的可读性，如果函数执行时不需要调用者传递数据则参数列表要写void，不要空着。

3、如果函数有返回值但没有写return语句，调用该函数时依然有返回值。
当调用一个有返回值的函数时，系统会为调用者和被调用者约定一个空间用于存储返回值，而return语句的作用就是把一个数据存储到这个空间，如果没有写return语句，调用者依然会从共用空间读取返回值，只是读取到的数据是随机的。
gcc -Wall -Werror xxx.c 可以防止漏写return语句。

4、当使用数组作为函数的参数传递时，它的长度信息就丢失了，(数组会蜕变成指针)，无法使用sizeof计算数组的长度，需要调用者额外提供一个参数作为数组的长度。

5、当使用二维数组作为函数的参数时，C语言规则定义二维数组时必须有列数，所以要行、列数在前，数组在后，并且把列数设置给数组。